Measuring Hash Table Collision Behavior

A separate chaining experiment using student ID keys

In this lab, you will measure how different bucket-selection rules distribute the same 50,000 student IDs across a hash table. You will make predictions before running the program, collect collision and comparison data, change the table capacity, and use graphs to support your explanation. The goal is to explain what the evidence shows, not simply to report which rule is fastest.

Lab at a glance

Course	Time	Language	Main evidence
CMPS 2131 Data Structures	About 3 hours	C23	Recorded outputs, measurement tables, two graphs, explanations

What you will learn

- Explain how a hash rule maps a key to a bucket and how separate chaining stores collisions.
- Compare distribution using buckets used, longest chain, average comparisons, and lookup time.
- Test how changing the number of buckets changes the observed collision behavior.
- Support a conclusion with reproducible measurements, tables, and graphs.

Files supplied

File	Purpose in this lab
measure_rules.c	Generates a fixed set of 50,000 IDs and measures four bucket-selection rules. This is the main experiment.
hash_table.h and hash_table.c	The separate-chaining hash table implementation, including insertion, lookup, deletion, growth, and invariant checking.
test_hash_table.c	Small correctness tests for collisions, updates, removal, and automatic table growth.

Important: the supplied implementation uses separate chaining. Each bucket points to a linked list of entries. The measurement program models those chains directly so you can compare how the four rules distribute the same keys.

The measurements

The program reports four measures. “Used” counts nonempty buckets. “Longest” is the largest chain. “Avg_cmp” is the average number of key comparisons required to find each stored key, assuming successful lookups. “Lookups_ms” is elapsed time for one pass that computes each key’s bucket and searches its chain. The last measure includes both hashing and chain traversal.

The load factor is α = N / M, where N is the number of keys and M is the number of buckets. At the default settings, N = 50,000 and M = 65,536. Keep this value in mind when comparing capacities.

1. Compile and run the programs

Put the four supplied files in one directory. Open a terminal, change into that directory, and confirm the files are there:

```
cd path/to/your/hash-lab
ls
```

Compile and run the measurement program. Compile it with optimization because you will record timing. The program does not need hash_table.c; it contains its own compact chain simulation.

```
gcc -std=c23 -Wall -Wextra -Wpedantic -O2 measure_rules.c -o measure_rules
./measure_rules
```

The first line of output gives N, M, and the load factor. Each following line reports one rule. Keep the full terminal output as evidence. Run the program five times at the default capacity so you can see how much the timing varies.

Run the correctness tests too

Compile the hash table implementation together with its test file, then run the test executable:

```
gcc -std=c23 -Wall -Wextra -Wpedantic -O2 test_hash_table.c hash_table.c -o test_hash_table
./test_hash_table
```

The expected final message is “all hash table tests passed”. If compilation fails, check that all three files are in the current directory, that your compiler supports C23, and that the command names match the files exactly. Record your compiler version with gcc --version.

Before you run: make predictions

Read the four rule functions in measure_rules.c. For each rule, predict whether it will use many or few buckets and whether its chains will tend to be short or long. Write a reason for each prediction. Do this before looking at the output.

Rule	Prediction: buckets and chains	Reason
year_rule	Few buckets very long chains	Students in a database enroll within a 4-6 year window meaning almost all 50,000 IDs will collide into just 4 to 6 buckets.
last4_rule	Many buckets medium chains	The last 4 digits offer 10,000 unique combinations (0000-9999). This will spread keys much better than the year, but will still cause collisions since N (50,000) > 10,000.
digit_sum_rule	Very few buckets, massive chains	The max possible sum of the digits in a standard ID is very small
fnv1a_rule	Many buckets, short chains	FNV-1a is designed to distribute bits evenly across the entire capacity of the table (65,536), minimizing collisions and keeping chains short.

2. Experiment A: compare the four rules

Run measure_rules five times using the supplied M = 65,536. The distribution statistics should be stable across runs; timing may vary. Keep all five complete terminal outputs as your raw timing record. In the table, copy the distribution measures from one run and calculate the median lookup time.

Rule	Used	Longest	Avg_cmp	Median ms

year				
last4				
digit-sum	41	3330	1280.94	(Run 5x to find)
fnv1a	33834	6	1.42	(Run 5x to find)

For each timing median, sort the five values from smallest to largest and use the middle value. Keep the five raw timings in your evidence notes; a median hides less of the run-to-run variation than a single selected time.

Graph the evidence

Create a bar graph of Avg_cmp for the four rules at M = 65,536. Label both axes and give the graph a descriptive title. If one value makes the others difficult to see, use a logarithmic vertical axis and label it clearly, or make a second graph for the smaller values. Do not omit any rule.

Interpret the comparison

- Which rule used the fewest buckets? Which had the longest chain?
- Does Avg_cmp agree with the chain lengths? Explain the relationship using one row of your data.
- Did the lookup-time ranking match the Avg_cmp ranking? If not, what else is included in lookup time?
- Which predictions were supported by the output? Identify evidence for each claim.

What the program measures

The code inserts each key at the head of its simulated chain. It then looks up every key once. For a chain of length L, finding all keys in that chain takes 1 + 2 + … + L comparisons, regardless of the order of the keys. The program sums this quantity across buckets and divides by N to produce Avg_cmp. This is a structural measure; lookup time is an elapsed-time measurement and can vary with the computer and other running programs.

3. Experiment B: change the bucket count

Now investigate the effect of table capacity. In measure_rules.c, find the enum near the top that defines N and M. Change only M. Keep the keys, N, and all four rules unchanged. Use M =

16,384 and M = 131,072 in addition to the original 65,536. Recompile after each change and save the output.

```
gcc -std=c23 -Wall -Wextra -Wpedantic -O2 measure_rules.c -o measure_rules
./measure_rules
```

Before each run, calculate α = 50,000 / M and predict how Avg_cmp and Longest will change for each rule. Record results from one run for the structural measures. Repeat each capacity five times only for timing, and record the median.

M	Load factor α	Rule	Used	Longest	Avg_cmp	Median ms
16,384		year				
16,384		last4				
16,384		digit-sum				
16,384		fnv1a				
65,536		year				
65,536		last4				
65,536		digit-sum				
65,536		fnv1a				
131,072		year				
131,072		last4				
131,072		digit-sum				
131,072		fnv1a				

Graph and explain the capacity experiment

Create a line graph with M on the horizontal axis and Avg_cmp on the vertical axis. Plot one line per rule. Use a logarithmic vertical axis if needed to make the differences visible, and label the scale. Include the three actual capacities and a legend.

- As M increased, which rules improved? Which changed little? Point to measured values.
- Why can two rules behave differently at the same N and M?
- Did a larger table guarantee short chains for every rule? Use your results to answer.
- How did the load factor help you predict the FNV-1a results? What did it fail to explain about the other rules?

4. Connect the measurements to the implementation

Use hash_table.c and test_hash_table.c to connect the experiment to the actual data structure. The measurement program isolates distribution. The implementation shows how a chained table performs its operations and maintains its invariants.

1. In hash_table.h, identify the array of bucket heads and the next pointer in an entry. Draw a small diagram of one bucket containing a chain of three entries.
2. In test_hash_table.c, find the collision test. It uses a table with eight buckets and keys that all map to bucket 1. Trace the chain after each of the three insertions.
3. In hash_table.c, trace ht_get to find the loop that walks through a chain. Then trace ht_remove and explain how it bypasses the removed entry.
4. Run the correctness tests. Explain what ht_check verifies and why passing these tests does not, by itself, prove that a hash rule distributes keys well.

Evidence and submission checklist

- Prediction table completed before the first run.
- Compiler version, compile commands, and complete terminal output for the baseline and both changed capacities.
- Measurement tables with N, M, load factor, used buckets, longest chain, Avg_cmp, and five raw timing runs with medians.
- Two labeled graphs: Avg_cmp by rule at the baseline capacity, and Avg_cmp by capacity for all four rules.
- A chain sketch for the collision test and written answers to the interpretation questions.
- A 3–5 minute recorded walkthrough showing the compile/run process, one graph, and an explanation supported by specific measurements.

Conclusion

Write a short conclusion (150–250 words) answering the lab question: How did the bucket-selection rule and the number of buckets affect collision behavior for this fixed set of IDs? Support the answer with at least three specific values from your tables, refer to both graphs, and distinguish comparison counts from elapsed time.

The choice of a bucket-selection rule has a huge impact on the amount of collisions in a hash-table, because it determines how well a hash-function can “distribute” the natural pattern of a fixed set of IDs. A rule like the sum of the characters in a ID (digit-sum) leads to very poor distribution, as shown in the data above: here only 41 out of 65,536 buckets were used to store IDs, leading to a longest chain of 3,330 and a structural cost of 1280.94 average comparisons. In the above table the huge capacity of the hash-table was not put to good use. A good hash-function like FNV-1a on the other hand ignores the IDs all together and distributes the bits as evenly as possible. The data above shows that FNV-1a uses 33,834 of the 65,536 buckets to store the IDs, forming only a longest chain of 6, while the average number of comparisons to search for an ID is near ideal at 1.42. Increasing the number of buckets M to distribute the IDs even better only helps reducing collisions if the hash-function is able to also map to these new buckets. The time it takes to perform lookups in a hash-table is heavily affected by the current load and the hardware of the system on which the program is running, so the structural quality of a hash-rule’s distribution is best measured by the average number of comparisons it takes to perform a lookup.

Evidence rule: do not write “the rule is good” or “the table is fast” without naming the measurement that supports the claim. Report what you observed, explain why the code could produce that result, and mention any limitation of the timing measurement.